Curso Avançado

Capítulo 9 - Integração entre os dois ESP32

Trocar comandos e informações mantendo uma autoridade comum para as decisões de circulação.

Introdução	Quando o sistema possui controladores distribuídos, cada um conhece apenas parte da montagem. A comunicação precisa preservar identidade, sequência, validade e confirmação, sem criar autorizações conflitantes.
Objetivo	Distinguir o coordenador da gaveta executora e definir como comandos, recebimentos e informações de estado participam da operação.
Meta de aprendizagem	Testar uma simulação de mensagens, rejeitar informações antigas e reconhecer por que receber um comando não comprova a posição de um equipamento.

1. Dois controladores como exemplo de integração

O título utiliza dois ESP32 para estudar uma ligação entre coordenador e executor. A arquitetura da EFMN 74 prevê controladores nas gavetas. A mesma lógica pode ser ampliada para GA1, GA2, GA3 e as funções de controle, conforme a distribuição vigente.

O coordenador decide sobre rotas e reservas. A gaveta executora acompanha seus sensores e equipamentos e informa resultados. Um executor não concede sozinho autorização para utilizar recursos compartilhados com outras gavetas.

Neste capítulo, todo o intercâmbio é simulado no Monitor Serial de um único controlador. O exemplo não configura rede, rádio ou ligação elétrica entre placas.

2. Comunicação precisa carregar significado

Uma mensagem deve permitir responder: quem enviou, a qual sessão pertence, qual informação contém e a qual comando ou evento se refere. Um texto como “pronto” não basta sem esse contexto.

3. Responsabilidades de cada lado

Função	Responsabilidade
Coordenador	Cadastro de operações, reserva dos recursos e autorização do percurso completo.
Executor da gaveta	Leitura local, execução de comandos permitidos e informação de estado ou falha.
Interface do operador	Solicitação de operação e apresentação das condições e avisos.
Proteção física	Atuação conforme o projeto, incluindo situações em que a comunicação falha.

Todos os comandos locais precisam respeitar os bloqueios válidos. Se houver controle manual no executor, sua utilização deve ser integrada à autoridade do coordenador e às regras da operação.

4. Conteúdo mínimo de uma mensagem

Campo	Finalidade
Origem e destino	Identificar os controladores e evitar aplicar a mensagem a outra gaveta.
Sessão ou inicialização	Distinguir a comunicação atual de informações anteriores.
Sequência	Reconhecer atualização nova, repetição ou atraso.
Tipo e recurso	Distinguir comando, recebimento, estado, evento ou falha.
Identificador do comando	Relacionar uma resposta à solicitação exata.
Dados e validade	Informar conteúdo, condição conhecida e necessidade de atualização.

Na implementação real, valide formato, tamanho, campos e integridade antes de usar os dados. A identificação do remetente também precisa corresponder ao dispositivo autorizado. O exemplo usa mensagens já interpretadas e não implementa essas verificações de transporte.

5. Recebimento não é confirmação mecânica

A resposta de recebimento, chamada ACK, informa que o comando foi reconhecido. Ela não comprova movimento do servo, posição das agulhas ou parada da composição. Essas condições exigem informações próprias, como discutido no capítulo 7.

6. Informação atualizada e perda de comunicação

O coordenador precisa saber se os dados necessários à decisão continuam atuais. Uma mensagem repetida não deve renovar o prazo como se fosse uma atualização nova. Uma mensagem de sessão antiga não deve reconstruir o estado atual.

Quando o executor reinicia, o coordenador deve invalidar as informações dependentes dele e interromper autorizações que não possam mais ser sustentadas. Reconectar a comunicação não comprova que a posição física continua igual.

7. Preparação de uma nova sessão

A sessão real deve ser estabelecida por uma troca controlada entre os dispositivos. Identifique as inicializações dos controladores, descarte respostas de operações anteriores e reconcilie os recursos antes de aceitar novos comandos.

O exemplo usa um contador local para distinguir sessões na mesma execução. Esse contador volta ao início quando a placa reinicia. Portanto, ele não é suficiente como identificador de inicialização para uma comunicação real.

8. Exemplo didático de mensagens

Comando do operador	Mensagem ou ação simulada
I	Inspecionar e iniciar nova sessão local. Nenhuma liberação de rota.
T	Gerar informação de estado com sequência nova.
D / O	Repetir a última sequência / enviar sessão antiga.
P	Emitir solicitação de preparação, se o estado estiver atual.
A / B	Gerar ACK do comando atual / ACK de comando diferente.
K	Conferir manualmente a posição física após o ACK.
X	Simular emergência e bloquear a comunicação operacional.

Há apenas um comando pendente no exercício. A confirmação de posição aparece separada do ACK. Mesmo após K, o programa não autoriza circulação nem libera reservas: essas decisões pertencem à lógica de rotas.

9. Código em C++ - sessão e informações de estado

Reúna as duas partes no mesmo sketch e use 115200 baud. As mensagens são produzidas pelo próprio exercício, sem segundo ESP32 conectado.

```
enum class Kind { STATUS, ACK };
struct Message {
  int source;
  unsigned long session;
  unsigned long sequence;
  Kind kind;
  unsigned long commandId;
};
const int executorId = 3;
unsigned long session = 0, lastSequence = 0, simulatedSequence = 0;
unsigned long commandCounter = 0, pendingId = 0, acceptedId = 0;
unsigned long lastStatusAt = 0, commandSentAt = 0;
const unsigned long statusTimeoutMs = 5000UL;
const unsigned long ackTimeoutMs = 3000UL;
bool blocked = true, statusKnown = false, positionConfirmed = false;

void blockLink(const char* reason) {
  blocked = true;
  statusKnown = positionConfirmed = false;
  Serial.println("SIMULATION: stop request. Reservations retained.");
  Serial.println(reason);
}
void receiveMessage(const Message &m) {
  if (blocked) { Serial.println("Blocked: packet ignored."); return; }
  if (m.source != executorId || m.session != session) {
    Serial.println("Wrong source or old session: ignored."); return;
  }
  if (m.sequence <= lastSequence) {
    Serial.println("Repeated or old sequence: ignored."); return;
  }
  lastSequence = m.sequence;
  if (m.kind == Kind::STATUS) {
    statusKnown = true;
    lastStatusAt = millis();
    Serial.println("Fresh simulated status. No position proof.");
  } else if (m.kind == Kind::ACK) {
    if (pendingId == 0 || m.commandId != pendingId) {
      Serial.println("Unrelated ACK: ignored."); return;
    }
    acceptedId = pendingId;
    pendingId = 0;
    Serial.println("Command accepted. Inspect position before K.");
  }
}
void simulateMessage(Kind kind, unsigned long id,
                     bool duplicate, bool oldSession) {
  unsigned long seq = duplicate ? simulatedSequence : ++simulatedSequence;
  Message m = {executorId, oldSession ? session - 1 : session,
               seq, kind, id};
  receiveMessage(m);
}
```

10. Código em C++ - solicitação, ACK e conferência

```
void handleCommand(char c) {
  if (c == 'X') { blockLink("Emergency"); return; }
  if (c == 'I' && blocked) {
    // Manual inspection and reconciliation of physical resources.
    // Real use also requires a peer session handshake.
    ++session;
    lastSequence = simulatedSequence = 0;
    pendingId = acceptedId = 0;
    statusKnown = positionConfirmed = false;
    blocked = false;
    Serial.println("New local session. Await T.");
    return;
  }
  if (blocked) { Serial.println("Inspect before I."); return; }
  if (c == 'T') simulateMessage(Kind::STATUS, 0, false, false);
  else if (c == 'D') simulateMessage(Kind::STATUS, 0, true, false);
  else if (c == 'O') simulateMessage(Kind::STATUS, 0, false, true);
  else if (c == 'P' && statusKnown && pendingId == 0 && acceptedId == 0) {
    pendingId = ++commandCounter;
    commandSentAt = millis();
    positionConfirmed = false;
    Serial.print("SIMULATION preparation command: ");
    Serial.println(pendingId);
  } else if (c == 'A') simulateMessage(Kind::ACK, pendingId, false, false);
  else if (c == 'B') simulateMessage(Kind::ACK, pendingId + 1, false, false);
  else if (c == 'K' && acceptedId != 0 && statusKnown) {
    // Operator checks the actual turnout position.
    positionConfirmed = true;
    Serial.println("Position manually checked. No route authorization.");
  } else blockLink("Unexpected command or unavailable data");
}
void setup() {
  Serial.begin(115200);
  blockLink("Startup: inspect before I.");
}
void loop() {
  unsigned long now = millis();
  if (!blocked && statusKnown && now - lastStatusAt >= statusTimeoutMs)
    blockLink("Status update timeout");
  if (!blocked && pendingId != 0 && now - commandSentAt >= ackTimeoutMs)
    blockLink("Command acknowledgement timeout");
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c != '\n' && c != '\r' && c != ' ') handleCommand(c);
  }
}
```

Envie T em intervalos menores que cinco segundos durante o teste. O ACK deve chegar antes de três segundos. Esses prazos são didáticos e não estão validados para uma rede real.

O contador de sequência não trata sua volta ao início após esgotar a faixa numérica. O protocolo real precisa definir esse caso e o reinício dos dispositivos. A simulação não substitui esse projeto.

11. Roteiro de testes

Teste	Resultado esperado
I T P A K	Estado atualizado, comando reconhecido e posição conferida separadamente.
I P	Sem T, preparação bloqueada por falta de informação.
I T P B	ACK não corresponde ao comando. Solicitação continua pendente.
I T P; aguardar 3 s sem A	Falha por ausência de ACK. Sem liberação de reservas.
I T D D; aguardar 5 s desde T	Repetições não renovam o prazo. Falha por estado antigo.
I T O	Sessão antiga ignorada. Informação válida anterior permanece até vencer.
I T P A; não enviar K	ACK recebido, mas posição continua sem confirmação.
X; depois T ou A	Bloqueio mantido. Mensagens não rearmam a operação.

Na sequência normal, envie os comandos dentro dos prazos. Depois de uma falha, confira a situação antes de I. O exercício inicia uma nova sessão de comunicação, mas não apaga reservas da lógica de rotas.

12. Integração com várias gavetas

Mantenha dados e prazos por executor. Uma atualização da GA1 não torna atuais as informações da GA2. O coordenador só autoriza uma rota quando todos os controladores e recursos envolvidos apresentam condições válidas.

No executor, repetir um comando já aceito não deve repetir indevidamente uma ação. Ele deve reconhecer o identificador e devolver a resposta correspondente. O exemplo estuda apenas o lado do coordenador; essa proteção também precisa existir no executor real.

Para escolher o transporte, considere distâncias, cabeamento, interferências, atualização e comportamento em falha. A implementação do meio de comunicação deve preservar as regras da operação e ser testada antes de acionar equipamentos.

13. Exercício e registro no Diário de Bordo

Liste as mensagens necessárias entre controle e GA3: sensores, preparação de desvios, recebimento, confirmação de posição e falha. Para cada uma, indique origem, destino, operação e condição de validade.

Teste atualização perdida, repetição, resposta antiga e reinício. Registre a reação e o procedimento de recuperação.